

Reviewer Pack — Roth Density of ACC⁰ Truth Tables Lower-Bounds Sipser Hardne...

Entry b80de3626c1d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 21:40:17 UTC

Roth Density of ACC⁰ Truth Tables Lower-Bounds Sipser Hardness

Entry ID: b80de3626c1d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-07 21:40:17 UTC

1. Conjecture

Field A (mathematical branch): Additive combinatorics: Roth-type 3-AP density in $\mathbb{Z}_2^{2^n}$ under binary integer encoding **Field B** (complexity object): ACC⁰[6] circuit lower bounds via the Williams 2014 framework; Sipser function as explicit candidate

Statement:

Encode $f: \{0,1\}^n \rightarrow \{0,1\}$ as $A_f \subseteq \mathbb{Z}_2^{2^n}$ by viewing each accepting input as its binary integer, and define the relative 3-AP density $R_3(f) := |\{(x,y,z) \in A_f^3 : x+z \equiv 2y \pmod{2^n}\}| \cdot 2^n / \max(|A_f|^3, 1)$ (so a uniformly random subset has $R_3 = 1 + o(1)$). Conjecture: every ACC⁰[6] circuit of depth $d \leq 4$ and size $s \leq n^{\log n}$ computing f satisfies $R_3(f) \geq 1 - (s/2^{n/(8d)})$, so for $n \geq 12$ every such circuit forces $R_3(f) \geq 0.8$; while the depth-3 Sipser function S_n (alternating AND-OR-AND with fan-in $\lceil n^{1/3} \rceil$) satisfies $R_3(S_n) \leq 0.6$ for all $n \geq 12$, certifying $S_n \notin \text{ACC}^0[6]$. Any single ACC⁰[6] sample with $R_3 < 0.8$ or any Sipser realization with $R_3 > 0.6$ falsifies the conjecture.

Rationale (proposer's reasoning):

R_3 measures sensitivity to additive structure under the canonical binary encoding, a coordinate the Razborov–Smolensky low-degree-polynomial machinery does not directly govern, so it offers a fresh handle inside the Williams algorithm-vs-lower-bound program. Crucially, ‘low R_3 ’ is a SPARSE property (random functions have $R_3 \approx 1$), so it is non-natural in the Razborov–Rudich sense, escaping the natural-proofs barrier listed safe by Williams while remaining constructive in time $O(2^{2n})$. The constructive map (truth table $\rightarrow$ integer support $\rightarrow$ 3-AP count) is fully explicit and avoids relativization/algebrization since it inspects the entire truth table rather than using oracle/polynomial-extension queries.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d35d66d7bf8259ff

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For $n \in \{12, 14\}$ across 30 seeds, every sampled $\text{ACC}^0[6]$ depth-3 size- n^2 circuit must yield $R_3(f) \geq 0.8$, and the Sipser S_n function must yield $R_3(S_n) \leq 0.6$. Support requires 100% of ACC^0 seeds satisfy the bound AND both Sipser instances satisfy the upper bound.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Roth 3-AP density Boolean function circuit lower bounds - additive combinatorics ACC^0 circuit lower bound truth table - Sipser function arithmetic progression Williams algorithmic method

Top relevant hits considered: - [s2:10.1109/FOCS.2019.00080] Why are Proof Complexity Lower Bounds Hard? - [s2:0f5716700eab6bfbf79f8f31d90a5bc5790641cf] Computing and combinatorics : 13th Annual International Conference, COCOON 2007 Banff, Canada, July 16-19, 2007 : procee

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def mod2(x):
        return x % 2

    def mod3(x):
        return x % 3

    def and_gate(a, b):
        return a and b

    def or_gate(a, b):
        return a or b

    def count_3AP(A_f, n):
        count = 0
```

```

    for x in range(2**n):
        for z in range(2**n):
            if (x + z) % 2 == 0:
                y = (x + z) // 2
                if y in A_f:
                    count += 1
    return count

def generate_truth_table(depth, size, n):
    gates = []
    for _ in range(size):
        gate_type = random.choice(['and', 'or'])
        if gate_type == 'and':
            inputs = [random.choice([mod2, mod3]) for _ in range(2)]
        else:
            inputs = [random.choice([mod2, mod3]) for _ in range(2)]
        gates.append((gate_type, inputs))
    return gates

def evaluate_circuit(circuit, n):
    truth_table = {}
    for i in range(2**n):
        input_bits = [(i >> j) & 1 for j in range(n)]
        output = input_bits[0]
        for gate_type, inputs in circuit:
            if gate_type == 'and':
                output = and_gate(output, inputs[0](input_bits[inputs[1]]))
            else:
                output = or_gate(output, inputs[0](input_bits[inputs[1]]))
        truth_table[i] = output
    return truth_table

def R3(truth_table, n):
    A_f = set(key for key, value in truth_table.items() if value == 1)
    count = count_3AP(A_f, n)
    max_count = len(A_f)**3
    return count * 2**n / max(max_count, 1)

def Sipser_function(n):
    fan_in = math.ceil(n**(1/3))
    truth_table = {}
    for i in range(2**n):
        input_bits = [(i >> j) & 1 for j in range(n)]
        output = input_bits[0]
        for j in range(fan_in - 1):
            if input_bits[j] == 1:
                output = or_gate(output, input_bits[j + 1])
            else:
                output = and_gate(output, input_bits[j + 1])
        truth_table[i] = output
    return truth_table

n_values = [10, 12, 14]
results = []

for n in n_values:

```

```

    for _ in range(30):
        circuit = generate_truth_table(3, n**2, n)
        truth_table = evaluate_circuit(circuit, n)
        R3_value = R3(truth_table, n)
        results.append(R3_value)

Sipser_R3 = R3(Sipser_function(14), 14) # Using n=14 for consistency

min_ACC0_R3 = min(results)
conjecture_holds = min_ACC0_R3 >= 0.8 and Sipser_R3 <= 0.6
counterexample = "" if conjecture_holds else "min ACC0 R3 < 0.8 or Sipser R3 > 0.6"

return {
    "metric_name": "R3",
    "metric_value": min_ACC0_R3,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_R3 = sum(result["metric_value"] for result in results) / len(results)
    std_R3 = math.sqrt(sum((result["metric_value"] - mean_R3)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_R3} std={std_R3} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"min ACC0 R3 < 0.8 or Sipser R3 > 0.6\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6cbf74e0.py", line 117, in <module>

```

    trial_result = run_trial(seed)
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6cbf74e0.py", line 93, in run_trial

```

    truth_table = evaluate_circuit(circuit, n)

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6cbf74e0.py", line 63, in evaluate_circ
    output = or_gate(output, inputs[0](input_bits[inputs[1]]))
```

```
TypeError: list indices must be integers or slices, not function
```

8. Critic adversarial review